

The Complete Docker & Kubernetes Guide: From Zero to DevSecOps

Who this guide is for: Someone who wants full, practical mastery of Docker and Kubernetes as the foundation for a DevSecOps career. No prior experience assumed. Every step is numbered, every concept is explained. Platform notes for both macOS and Linux are integrated throughout.

Table of Contents

1. The Big Picture — Why Docker and Kubernetes?
2. Part 1 — Docker: Containerizing Everything
 - 2.1 What is a Container? (The Real Explanation)
 - 2.2 Installing Docker (macOS & Linux)
 - 2.3 Your First Container — Hello World
 - 2.4 Docker Images — Building Blocks
 - 2.5 Writing Your First Dockerfile
 - 2.6 Docker Volumes — Persisting Data
 - 2.7 Docker Networks — Containers Talking to Each Other
 - 2.8 Docker Compose — Running Multi-Container Apps
 - 2.9 Dockerfile Best Practices
 - 2.10 Docker Security Basics
3. Part 2 — Kubernetes: Orchestrating Everything
 - 3.1 What is Kubernetes? (The Real Explanation)
 - 3.2 Setting Up a Local Cluster (kind — macOS & Linux)
 - 3.3 Core Concepts: Pods, Nodes, Clusters
 - 3.4 Deployments — Managing Your App
 - 3.5 Services — Exposing Your App
 - 3.6 ConfigMaps and Secrets — Configuration Done Right
 - 3.7 Volumes and Persistent Storage
 - 3.8 StatefulSets — Databases in Kubernetes
 - 3.9 Health Checks: Liveness, Readiness, and Startup Probes
 - 3.10 Rolling Updates and Rollbacks
 - 3.11 Horizontal Pod Autoscaler (HPA) — Auto-Scaling

- 3.12 Networking Deep Dive — Ingress and Network Policies
- 3.13 RBAC — Security and Access Control
- 3.14 Helm — The Kubernetes Package Manager

4. [Part 3 — Real-World Project: Deploying a Full App](#)

5. [Part 4 — DevSecOps Context: Where Security Fits In](#)

6. [Quick Reference Cheat Sheet](#)

1. The Big Picture

Why Should You Care?

Imagine you build a web application on your laptop. It works perfectly. You send it to a friend, and it breaks. Why? Because their computer has a different version of Python, different libraries, different settings. This is the classic "it works on my machine" problem.^{[1][2]}

Now scale that problem up. A real company might run 50 different services — a login service, a payment service, a notification service, a database. Each has different language requirements, different versions, different dependencies. Deploying and managing all of these on bare servers manually is a nightmare.^{[3][4]}

Docker solves the "it works on my machine" problem. It packages your app and everything it needs into a single portable unit called a container.^{[2][1]}

Kubernetes solves the "how do I manage hundreds of containers" problem. It is a platform that automatically deploys, scales, and keeps your containers running.^{[5][6][^7]}

The DevSecOps Path

DevSecOps = Development + Security + Operations. The idea is that security is not something you bolt on at the end — it is baked into every step of building and running software.^{[4][3]}

The pathway looks like this:

- **Docker** → You learn to package and run applications as containers
- **Kubernetes** → You learn to run and manage those containers at scale
- **Security layer** → You apply security at every step (image scanning, RBAC, network policies, secrets management)
- **CI/CD layer** → You automate building, testing, and deploying securely

This guide covers the first two in depth, with security woven throughout.^{[8][3][^4]}

Part 1 — Docker

2.1 What Is a Container? (The Real Explanation)

The Analogy

Think of a shipping container. Before shipping containers existed, loading cargo onto a ship was a mess — boxes of different sizes, crates, loose items. Everything had to be handled differently. Then someone invented the standard steel shipping container. Now it doesn't matter if you're shipping bananas or computers — everything goes into the same standardized box, and every ship, truck, and crane knows how to handle it.^{[1][2]}

A Docker container is the same idea for software. It is a standardized box that contains:

- Your application code
- All the libraries your code depends on
- The runtime (e.g., Python 3.11 or Node.js 20)
- Configuration files
- Basically, everything needed to run the app

And it works identically on your laptop, a colleague's machine, a test server, or a production cloud.^{[2][1]}

Container vs. Virtual Machine

A common question: "How is this different from a Virtual Machine (VM)?"

	Container	Virtual Machine
Size	Usually 50MB–500MB	Usually 5GB–50GB
Start time	Seconds or less	Minutes
What it includes	App + libraries + runtime	Full OS + app + libraries
Isolation	Process-level isolation	Full hardware-level isolation
Use case	Running many microservices	Running completely separate systems

A VM is like renting an entire house. A container is like renting a single room in a shared house — the plumbing and electricity (the host OS kernel) are shared, but your room (the container) is yours.^{[1][2]}

Key Terms Before You Start

- **Image** — A blueprint or recipe. It describes what the container should contain. It's read-only.^[^1]
- **Container** — A running instance of an image. You can have many containers running from the same image.^[^1]

- **Docker Hub** — A public registry (like GitHub but for images) where official images live.^{[2][1]}
- **Dockerfile** — A text file with instructions for building your own image.^[^9]

2.2 Installing Docker (macOS & Linux)

macOS Installation

Docker on macOS runs through **Docker Desktop**, which provides a graphical UI as well as the command-line tools you'll use.^{[10][1]}

Step 1 — Check your chip type.

Open Terminal and run:

```
uname -m
```

- If it says `x86_64` — you have an Intel Mac.
- If it says `arm64` — you have an Apple Silicon Mac (M1/M2/M3/M4).

This matters because you need to download the correct version.

Step 2 — Install using Homebrew (recommended).

If you don't have Homebrew installed, install it first:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then install Docker Desktop:

```
brew install --cask docker
```

Step 3 — Launch Docker Desktop.

Open your Applications folder and double-click Docker. You'll see a small whale icon appear in your menu bar. Wait for it to say "Docker Desktop is running."

Step 4 — Verify in Terminal.

```
docker --version  
docker compose version
```

You should see output like:

```
Docker version 27.x.x, build xxxxxxxx  
Docker Compose version v2.x.x
```

If you see this, Docker is installed and working. ✓

Linux Installation (Ubuntu/Debian)

On Linux, you install Docker Engine directly — no GUI needed.^[11]_[1]

Step 1 — Update your package list and install prerequisites.

```
sudo apt update
sudo apt install ca-certificates curl gnupg
```

What this does: `apt update` refreshes the list of available software. The other packages are needed to securely add Docker's repository.

Step 2 — Add Docker's official GPG key.

A GPG key lets your system verify that packages actually come from Docker and haven't been tampered with.

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Step 3 — Add Docker's repository.

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] ht
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

What this does: Adds Docker's repository to your system so `apt` can find Docker's packages.

Step 4 — Install Docker.

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-c
```

Step 5 — Add your user to the Docker group.

By default, Docker commands require `sudo`. This step lets you run Docker without it:

```
sudo usermod -aG docker $USER
newgrp docker
```

What this does: Adds your current user to the `docker` group. `newgrp docker` applies the change immediately without needing to log out.

Step 6 — Verify installation.

```
docker --version
docker compose version
```

Other Linux Distributions

- **Arch Linux / Manjaro / CachyOS:**

```
sudo pacman -S docker docker-compose
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

- **Fedora / RHEL:**

```
sudo dnf install docker-ce docker-ce-cli containerd.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

- **Parrot OS / BlackArch (Debian-based):** Follow the Ubuntu/Debian steps above.

2.3 Your First Container — Hello World

Now that Docker is installed, let's run your first container.

Step 1 — Pull and run a test image.

```
docker run hello-world
```

You'll see something like:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

What just happened?

1. Docker looked for an image called `hello-world` on your machine. It wasn't there.
2. Docker automatically went to Docker Hub (the public registry) and downloaded the image.
3. Docker created a container from that image and ran it.
4. The container printed its message and exited.

Step 2 — Run a real web server.

Let's run nginx (a popular web server) as a container:

```
docker run -d -p 8080:80 --name my-nginx nginx
```

Breaking down the flags:

- `-d` — "detached mode." Runs the container in the background so you get your terminal back.
- `-p 8080:80` — Port mapping. "Take the container's port 80 and expose it as port 8080 on my machine." Format: `HOST_PORT:CONTAINER_PORT`
- `--name my-nginx` — Give this container a human-readable name.
- `nginx` — The image to use.

Now open your browser and go to `http://localhost:8080`. You should see the nginx welcome page. You just ran a web server in under 5 seconds.[^1]

Step 3 — Useful container commands.

```
# List running containers
docker ps

# List ALL containers (including stopped ones)
docker ps -a

# View logs from a container
docker logs my-nginx

# Follow logs in real time
docker logs -f my-nginx

# Stop a container
docker stop my-nginx

# Start a stopped container
docker start my-nginx

# Remove a container (must be stopped first)
docker rm my-nginx

# Stop and remove in one command
docker rm -f my-nginx

# Execute a command inside a running container (like SSH-ing in)
docker exec -it my-nginx bash
```

The `docker exec -it my-nginx bash` command is extremely useful. It opens an interactive shell (`-it` means interactive + pseudo-TTY) inside the running container. You can explore its filesystem, check files, run commands — great for debugging.[^2]

Step 4 — Explore the container from inside.

```
docker exec -it my-nginx bash
# Inside the container:
ls /etc/nginx/
cat /etc/nginx/nginx.conf
exit
```

2.4 Docker Images — Building Blocks

What Is an Image?

An image is like a snapshot or a recipe. When you run `docker run nginx`, Docker downloads the nginx image which contains everything needed: the nginx binary, its config files, and a minimal Linux filesystem.^{[2][1]}

Images are made up of **layers**. Each instruction in a Dockerfile creates one layer. Layers are cached and reused, making builds fast.^{[12][13]}

Working with Images

```
# List images on your machine
docker images

# Pull an image without running it
docker pull ubuntu:22.04

# See detailed information about an image
docker inspect ubuntu:22.04

# Remove an image
docker rmi ubuntu:22.04

# Remove all unused images (cleanup)
docker image prune -a
```

Image Tags

Images have tags that identify versions. `nginx` and `nginx:latest` are the same — `latest` is the default tag. But you should always use specific tags in real projects so you always get the same image.^{[13][14]}

```
docker pull nginx:1.25          # Specific version
docker pull node:20-alpine      # Node.js 20 on Alpine Linux (small image)
docker pull python:3.11-slim    # Python 3.11 slim variant
```


2.5 Writing Your First Dockerfile

A Dockerfile is a plain text file (literally named `Dockerfile`, no extension) that contains step-by-step instructions for building a custom image.^[^9]

Your First Real Application

Let's build a simple Python Flask web application and containerize it.

Step 1 — Create a project folder.

```
mkdir my-flask-app
cd my-flask-app
```

Step 2 — Create the application files.

Create `app.py`:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return '<h1>Hello from my Docker container!</h1>'

@app.route('/health')
def health():
    return {'status': 'ok'}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Create `requirements.txt`:

```
flask==3.0.0
```

Step 3 — Create the Dockerfile.

Create a file named exactly `Dockerfile` (no extension):

```
# Start from an official Python image
# Using 3.11-slim: smaller than full Python image
FROM python:3.11-slim

# Set the working directory inside the container
# All subsequent commands run from /app
WORKDIR /app

# Copy requirements first (for Docker layer caching)
# This layer only re-runs if requirements.txt changes
```

```
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code
COPY . .

# Create a non-root user for security
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
USER appuser

# Tell Docker this container listens on port 5000
EXPOSE 5000

# The command to run when the container starts
CMD ["python", "app.py"]
```

Why the ORDER matters: Docker caches each layer. If you copy `requirements.txt` first and install dependencies, that layer is cached. The next time you build (when you only change `app.py`), Docker reuses the cached dependencies layer and only re-runs the `COPY . .` step. This makes rebuilds very fast.^{[15][13]}

Step 4 — Build the image.

```
docker build -t my-flask-app:1.0 .
```

- `-t my-flask-app:1.0` — Tag the image with name `my-flask-app` and version `1.0`
- `.` — The build context is the current directory

You'll see Docker execute each instruction, one by one.

Step 5 — Run the container.

```
docker run -d -p 5000:5000 --name flask-app my-flask-app:1.0
```

Visit `http://localhost:5000` in your browser. You should see "Hello from my Docker container!"

Step 6 — Test the health endpoint.

```
curl http://localhost:5000/health
# Output: {"status":"ok"}
```

Understanding Dockerfile Instructions

Instruction	What It Does	Example
FROM	Sets the base image to build on	FROM python:3.11-slim
WORKDIR	Sets the working directory	WORKDIR /app
COPY	Copies files from host to image	COPY . .
RUN	Executes a command during build	RUN pip install flask
ENV	Sets environment variables	ENV PORT=5000
EXPOSE	Documents which port the app uses	EXPOSE 5000
CMD	Default command to run	CMD ["python", "app.py"]
ENTRYPOINT	Fixed executable for the container	ENTRYPOINT ["gunicorn"]
USER	Sets which user to run as	USER appuser
ARG	Build-time variable	ARG VERSION=1.0

CMD vs ENTRYPOINT:

- CMD can be overridden when running the container: `docker run my-app python other_script.py`
- ENTRYPOINT cannot be overridden easily — it's the core executable
- A common pattern: `ENTRYPOINT ["gunicorn"] + CMD ["app:app", "--workers", "4"]`

2.6 Docker Volumes — Persisting Data

The Problem

Containers are ephemeral (temporary) by design. When you remove a container, all data written inside it disappears. If you run a database container and store data inside the container's filesystem, that data is gone when the container is removed.^{[16][17][18]}

The Solution: Volumes

A volume is a folder on your **host machine** that is mounted into the container. Data written to the mounted path inside the container is actually written to your host machine. When the container is removed, the data stays on the host.^{[17][18]}

Types of Volumes

1. Named Volumes (Recommended)

Docker manages where the data is stored. You just give it a name.

```
# Create a named volume
docker volume create my-data

# Run a container with the volume mounted at /data
docker run -d \
  -v my-data:/data \
  --name test-container \
  ubuntu sleep infinity

# Write some data into the container
docker exec test-container bash -c "echo 'Hello, Volume!' > /data/hello.txt"

# Remove the container
docker rm -f test-container

# Run a new container with the SAME volume – data is still there!
docker run --rm \
  -v my-data:/data \
  ubuntu cat /data/hello.txt
# Output: Hello, Volume!
```

2. Bind Mounts (For Development)

You specify the exact path on your host. Useful for development when you want code changes to reflect instantly in the container.^[18]^[17]

```
# Mount your current directory into the container
docker run -d \
  -p 5000:5000 \
  -v $(pwd):/app \
  my-flask-app:1.0
```

Now if you edit `app.py` on your host, the container sees the change immediately (assuming your app reloads, e.g., with `flask run --reload`).

3. Real-World Example: PostgreSQL Database

This is a very common use case — running a database in Docker with persistent storage:

```
# Create a volume for database data
docker volume create postgres-data

# Run PostgreSQL with persistent storage
docker run -d \
  --name my-postgres \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_DB=myapp \
  -p 5432:5432 \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:16
```

```
# Connect to it
docker exec -it my-postgres psql -U postgres -d myapp
```

Even if you remove and recreate the container, your database data survives because it's stored in the postgres-data volume.^{[17][18]}

Volume Management Commands:

```
docker volume ls          # List all volumes
docker volume inspect my-data # Detailed info about a volume
docker volume rm my-data   # Remove a volume
docker volume prune        # Remove all unused volumes
```

2.7 Docker Networks — Containers Talking to Each Other

The Problem

If you run a web app container and a database container separately, the web app needs to connect to the database. By default, each container is isolated — they can't reach each other.^[2]
^[1]

Docker Networks to the Rescue

Docker networks allow containers to communicate with each other securely, without exposing ports to the outside world.^[^1]

Types of Networks

- **bridge** (default) — Containers on the same bridge network can communicate using container names as hostnames.
- **host** — Container uses the host's network directly. No isolation. (Linux only)
- **none** — No network. Fully isolated.

Hands-On: Connecting Flask to PostgreSQL

Step 1 — Create a custom network.

```
docker network create my-app-network
```

Step 2 — Start the database on the network.

```
docker run -d \
  --name my-postgres \
  --network my-app-network \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e POSTGRES_DB=myapp \
```

```
-v postgres-data:/var/lib/postgresql/data \
postgres:16
```

Step 3 — Start the web app on the same network.

```
docker run -d \
  --name my-flask \
  --network my-app-network \
  -p 5000:5000 \
  my-flask-app:1.0
```

Step 4 — Test connectivity.

```
docker exec -it my-flask bash
# Inside the container:
ping my-postgres
# Or connect to the database using "my-postgres" as the hostname!
```

Why does my-postgres work as a hostname? Docker's built-in DNS resolves container names to their internal IP addresses — but only within the same network.^[1]

Network Commands:

```
docker network ls                # List networks
docker network inspect my-app-network # See which containers are on the network
docker network connect my-app-network my-container # Add a container to a network
docker network disconnect my-app-network my-container # Remove from network
docker network rm my-app-network    # Remove network
```

2.8 Docker Compose — Running Multi-Container Apps

The Problem

Once you have a web app, a database, a Redis cache, and a message queue, running each with long `docker run` commands becomes unwieldy and error-prone.^{[19][20]}

Docker Compose: One File to Rule Them All

Docker Compose lets you define all your services, networks, and volumes in a single `docker-compose.yml` file. You then start everything with one command.^{[20][19]}

Real-World Example: Web App + Database + Redis

Create a folder called `my-fullstack-app` and set up this structure:

```
my-fullstack-app/
├─ docker-compose.yml
```

```
├─ app/
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
```

app/requirements.txt:

```
flask==3.0.0
redis==5.0.0
psycpg2-binary==2.9.9
```

app/app.py:

```
from flask import Flask
import redis
import os

app = Flask(__name__)
cache = redis.Redis(host='redis', port=6379)

@app.route('/')
def hello():
    count = cache.incr('visits')
    return f'<h1>Hello! You are visitor #{count}</h1>'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

app/Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

docker-compose.yml:

```
version: '3.8'

services:
  # The web application
  web:
    build: ./app          # Build from Dockerfile in the ./app folder
    ports:
      - "5000:5000"       # Expose port 5000
    environment:
      - FLASK_ENV=development
    depends_on:
```

```

    - redis          # Start after redis is ready
    - postgres
networks:
    - app-network

# Redis cache
redis:
    image: redis:7-alpine # Use official Redis image, Alpine variant (small)
    networks:
        - app-network

# PostgreSQL database
postgres:
    image: postgres:16
    environment:
        POSTGRES_DB: myapp
        POSTGRES_USER: myuser
        POSTGRES_PASSWORD: mysecretpassword
    volumes:
        - postgres-data:/var/lib/postgresql/data # Persist database data
    networks:
        - app-network

networks:
    app-network:
        driver: bridge

volumes:
    postgres-data:

```

Running the stack:

```

# Start everything
docker compose up

# Start in background (detached)
docker compose up -d

# View logs from all services
docker compose logs -f

# View logs from just the web service
docker compose logs -f web

# Stop everything
docker compose down

# Stop and remove volumes too (deletes database data!)
docker compose down -v

# Rebuild images and start
docker compose up -d --build

```


Now open `http://localhost:5000` — you'll see a visit counter that increments because it's stored in Redis.^{[19][20]}

Docker Compose Commands You'll Use Daily

```
docker compose ps          # List running services
docker compose exec web bash # Open a shell inside the 'web' service
docker compose restart web  # Restart a specific service
docker compose stop         # Stop without removing containers
docker compose start        # Start stopped containers
docker compose pull         # Pull latest images for all services
```

2.9 Dockerfile Best Practices

This section is important for DevSecOps — writing good Dockerfiles affects both security and efficiency.^{[21][12][^13]}

Best Practice 1: Use Official, Versioned Base Images

Bad:

```
FROM ubuntu
RUN apt install python3
```

Good:

```
FROM python:3.11-slim
```

Why: Official images are tested and maintained. `slim` and `alpine` variants are smaller and have fewer vulnerabilities.^{[14][13]}

Best Practice 2: Never Use `latest` in Production

Bad:

```
FROM nginx:latest
```

Good:

```
FROM nginx:1.25.3
```

Why: `latest` can break silently when a new major version is released.^{[13][14]}

Best Practice 3: Order Layers for Caching

Put things that change rarely at the top, things that change frequently at the bottom.

Slow (rebuilds dependencies every time you change code):

```
COPY . .  
RUN pip install -r requirements.txt
```

Fast (dependencies are cached, only re-run when requirements.txt changes):

```
COPY requirements.txt .  
RUN pip install -r requirements.txt  
COPY . .
```

Best Practice 4: Run as Non-Root User

By default, processes inside containers run as root. If a hacker exploits your app, they get root inside the container — bad.^{[14][13]}

```
RUN addgroup --system appgroup && \  
    adduser --system --ingroup appgroup --no-create-home appuser  
USER appuser
```

Best Practice 5: Use Multi-Stage Builds

Multi-stage builds produce tiny final images by keeping build tools out of the final image. Think of it like building a house: you use scaffolding during construction, but you don't leave the scaffolding in the final house.^{[12][13]}

```
# Stage 1: Build  
FROM node:20 AS builder  
WORKDIR /app  
COPY package*.json ./  
RUN npm install  
COPY . .  
RUN npm run build          # Produces the ./dist folder  
  
# Stage 2: Final (tiny image — no Node.js build tools!)  
FROM nginx:1.25-alpine  
COPY --from=builder /app/dist /usr/share/nginx/html  
EXPOSE 80
```

The final image only contains nginx and your built files — not the Node.js build tools. This can reduce image size from 1.2GB to under 50MB.^{[13][12]}

Best Practice 6: Use `.dockerignore`

Create a `.dockerignore` file to exclude files from the build context (similar to `.gitignore`):

```
node_modules/  
.git/  
*.log  
.env  
.DS_Store  
__pycache__/  
*.pyc
```

This speeds up builds and prevents accidentally including secrets in your image.^{[12][13]}

Best Practice 7: Never Store Secrets in Images

Never do this:

```
ENV DB_PASSWORD=mysupersecretpassword
```

Anyone who gets the image can extract that password with `docker inspect`. Pass secrets at runtime through environment variables, Docker secrets, or Kubernetes secrets.^[15]

2.10 Docker Security Basics

Since you're aiming for DevSecOps, security thinking starts here.^{[3][21]}

Principle of Least Privilege

- Run containers as non-root (covered above)
- Drop unnecessary Linux capabilities: `docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx`
- Use read-only filesystem: `docker run --read-only nginx`

Scanning Images for Vulnerabilities

Docker has built-in scanning:

```
# Scan an image for known vulnerabilities  
docker scout cves my-flask-app:1.0
```

Or use Trivy (a popular security scanner):

```
# macOS  
brew install trivy  
  
# Linux
```

```
curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh  
  
# Scan your image  
trivy image my-flask-app:1.0
```

Keep Images Updated

Rebuild images regularly to pick up security patches from the base image. A container is only as secure as its base image.^{[15][13]}

Part 2 — Kubernetes

3.1 What Is Kubernetes? (The Real Explanation)

The Problem Docker Alone Doesn't Solve

Docker is great for running one container or a handful of containers on one machine. But real applications might have 100, 1000, or 10,000 containers running across dozens of servers. Questions arise:^{[6][7][^5]}

- What happens when a container crashes at 3am? Who restarts it?
- What if your app gets popular and needs 50 more copies to handle the traffic?
- What if a server dies? How do you move containers off it automatically?
- How do you deploy a new version without downtime?
- How do you manage secrets, certificates, and config across all these containers?

Kubernetes (often abbreviated K8s — the 8 represents the 8 letters between K and s) answers all of these questions.^{[7][6]}

The Restaurant Analogy

Think of Kubernetes as the management system for a large restaurant chain:

- **You (the developer)** are the executive chef. You write the menu (YAML files) that says "I want 5 portions of this dish, made exactly this way."
- **Kubernetes** is the restaurant manager. It reads your menu and makes sure 5 portions are always ready. If one goes cold, it throws it out and makes a new one.
- **Nodes** are the individual kitchen stations (servers/machines).
- **Pods** are the individual dishes being prepared (running containers).
- The manager doesn't care which station makes the dish — it just ensures the desired count is always met.^{[6][7]}

Declarative vs. Imperative

This is the most important mental shift when learning Kubernetes.

Imperative (how most things work): "Do this. Then do that. Now do this."

- "Start 3 containers. If one dies, start another."

Declarative (how Kubernetes works): "This is what the world should look like. You figure it out."

- "I want 3 replicas of this container always running."

You describe the **desired state** in a YAML file. Kubernetes' job is to **reconcile** the actual state of the cluster to match your desired state, continuously and automatically.^{[5][7][^6]}

Kubernetes Architecture (Simplified)

A Kubernetes cluster has two types of machines:

Control Plane (the brain):

- **API Server** — Everything talks to this. It's the front door to Kubernetes.
- **etcd** — A key-value database that stores the entire cluster state.
- **Scheduler** — Decides which Node to put each Pod on.
- **Controller Manager** — Watches the cluster and fixes things (e.g., restarts crashed Pods).

Worker Nodes (the muscles):

- **kubelet** — An agent that runs on each node, receives instructions from the control plane, and manages containers on that node.
- **kube-proxy** — Manages networking rules so containers can communicate.
- **Container Runtime** — Actually runs containers (e.g., containerd).^{[7][6]}

3.2 Setting Up a Local Cluster with kind

kind (Kubernetes IN Docker) runs a full Kubernetes cluster inside Docker containers. It's the best tool for local development and learning.^{[22][23][^24]}

Why kind?

- Works on macOS and Linux
- Creates a full multi-node cluster
- Fully scriptable — you can automate cluster creation
- Used by the Kubernetes project itself for testing^{[23][22]}

Installing kind

macOS:

```
brew install kind
```

Linux:

```
# For x86_64 (Intel/AMD)
[ $(uname -m) = x86_64 ] && curl -Lo ./kind \
  https://kind.sigs.k8s.io/dl/v0.27.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind

# Verify
kind --version
```

Installing kubectl

kubectl is the command-line tool for talking to any Kubernetes cluster.^{[24][6]}

macOS:

```
brew install kubectl
kubectl version --client
```

Linux:

```
# Download the binary
curl -LO "https://dl.k8s.io/release/$(curl -L -s \
  https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

# Make it executable and install
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# Verify
kubectl version --client
```

Creating Your First Cluster

Step 1 — Create a single-node cluster.

```
kind create cluster --name my-cluster
```

Watch the output. Kind is:

1. Pulling a Docker image that contains all Kubernetes components

2. Starting a Docker container that acts as your Kubernetes node

3. Configuring your ~/.kube/config file so kubectl knows how to connect

Step 2 – Verify the cluster is running.

```
# Check cluster info
kubectl cluster-info --context kind-my-cluster

# See the nodes in the cluster
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
my-cluster-control-plane	Ready	control-plane	1m	v1.32.x

One node that acts as both the control plane and a worker. Perfect for learning.

Step 3 – Create a multi-node cluster (better for realistic practice).

Create a file called kind-config.yaml:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: dev-cluster
nodes:
- role: control-plane
- role: worker
- role: worker
```

Create the cluster:

```
kind create cluster --config kind-config.yaml
```

Verify:

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE
dev-cluster-control-plane	Ready	control-plane	2m
dev-cluster-worker	Ready	<none>	1m
dev-cluster-worker2	Ready	<none>	1m

Managing Multiple Clusters:

```
# List all clusters
kind get clusters

# List kubectl contexts (each cluster has one)
kubectl config get-contexts

# Switch to a specific cluster
kubectl config use-context kind-dev-cluster

# Delete a cluster
kind delete cluster --name my-cluster
```

3.3 Core Concepts: Pods, Nodes, Clusters

Namespace — The First Organizational Unit

Think of namespaces as folders in a filesystem. They let you organize your Kubernetes resources and apply access controls per team or environment.^{[25][6]}

```
# List existing namespaces
kubectl get namespaces
# You'll see: default, kube-system, kube-public, kube-node-lease

# Create a new namespace
kubectl create namespace my-app

# Set a default namespace for your current session
kubectl config set-context --current --namespace=my-app

# List resources in a specific namespace
kubectl get pods -n kube-system
```

kube-system is where Kubernetes' own infrastructure pods live. Never delete things from there.

Pods — The Smallest Unit

A Pod is the smallest deployable unit in Kubernetes. Think of a Pod as a wrapper around one or more containers that are always deployed together and share the same network and storage.^{[26][6]}

In 99% of cases, a Pod contains exactly one container. The multi-container Pod pattern is for sidecars (e.g., a log collector running alongside your app).

Creating your first Pod (declarative YAML):

Create a file called `my-pod.yaml`:

```
apiVersion: v1          # Which Kubernetes API version to use
kind: Pod               # The type of resource
```



```

metadata:
  name: my-nginx-pod      # Name of this pod
  namespace: default
  labels:                 # Labels are key-value pairs used to select resources
    app: nginx
    tier: frontend
spec:
  containers:
  - name: nginx           # Name of the container inside the pod
    image: nginx:1.25     # Which Docker image to use
    ports:
    - containerPort: 80   # Document that port 80 is used
  resources:              # Resource limits (good practice)
    requests:
      memory: "64Mi"
      cpu: "250m"
    limits:
      memory: "128Mi"
      cpu: "500m"

```

Understanding resource units:

- Memory: 64Mi = 64 Mebibytes, 1Gi = 1 Gibibyte
- CPU: 250m = 250 millicores = 0.25 CPU cores, 1 = 1 full CPU core

Apply the YAML:

```
kubectl apply -f my-pod.yaml
```

Check the Pod:

```

# List pods
kubectl get pods

# Get more details
kubectl get pods -o wide    # Shows which node it's on, IP address

# Get full details (events, conditions, etc.)
kubectl describe pod my-nginx-pod

# Get the YAML of a running pod
kubectl get pod my-nginx-pod -o yaml

# View logs
kubectl logs my-nginx-pod

# Open a shell inside the pod
kubectl exec -it my-nginx-pod -- bash

# Delete the pod
kubectl delete pod my-nginx-pod

```

```
# Or delete using the file
kubectl delete -f my-pod.yaml
```

Important: If you delete a Pod created directly (not via a Deployment), it's gone. It won't be recreated. This is why in practice, you always use Deployments, not raw Pods.

3.4 Deployments — Managing Your App

A **Deployment** is what you actually use in real life. You tell the Deployment: "I want N replicas of this Pod." The Deployment creates a **ReplicaSet**, which in turn creates the Pods. If a Pod crashes, the ReplicaSet controller detects it and creates a replacement automatically.^{[27][28]}
[^26]

Think of it this way:

- You write a job description (Deployment YAML)
- Kubernetes hires the workers (creates Pods)
- If a worker quits (Pod crashes), Kubernetes automatically hires a replacement

Creating a Deployment

Create `nginx-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3                # I want 3 copies of this pod always running

  selector:
    matchLabels:
      app: nginx            # This deployment manages pods with this label

  template:                 # Pod template — defines what each pod looks like
    metadata:
      labels:
        app: nginx         # MUST match selector.matchLabels
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
            limits:
```

```
memory: "128Mi"  
cpu: "200m"
```

Apply it:

```
kubectl apply -f nginx-deployment.yaml
```

Verify:

```
kubectl get deployments  
kubectl get pods           # You should see 3 nginx pods  
kubectl get replicaset     # Shows the ReplicaSet created by the deployment
```

Test Self-Healing:

Delete one of the pods by hand:

```
# Get the pod name  
kubectl get pods  
# Copy one of the nginx pod names, then delete it:  
kubectl delete pod nginx-deployment-XXXXX-XXXXX
```

Immediately run:

```
kubectl get pods
```

You'll see the deleted Pod terminating, and a new one spinning up instantly. Kubernetes noticed the actual state (2 pods) didn't match the desired state (3 pods) and fixed it.^[28]^[27]

Scaling:

```
# Scale to 5 replicas  
kubectl scale deployment nginx-deployment --replicas=5  
  
# Or edit the YAML and re-apply  
# Change replicas: 3 to replicas: 5, then:  
kubectl apply -f nginx-deployment.yaml  
  
# Check the scaling  
kubectl get pods
```

3.5 Services — Exposing Your App

Pods have IP addresses, but those IPs change every time a Pod restarts. A **Service** provides a stable, permanent endpoint that always routes traffic to healthy Pods.^{[29][28][^26]}

Think of a Service as a load balancer. The Pods behind it can come and go, but the Service's IP and DNS name stays the same.

Types of Services

Type	Description	When to Use
ClusterIP	Only accessible inside the cluster	Backend services, databases
NodePort	Exposes on a port on each Node's IP	Development, testing
LoadBalancer	Creates external load balancer (cloud)	Production on cloud providers
ExternalName	Maps to external DNS name	Connecting to external databases

Creating a Service

Create `nginx-service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx          # Routes traffic to pods with this label
  ports:
    - protocol: TCP
      port: 80           # Port the Service listens on
      targetPort: 80     # Port the Pods are listening on
      type: ClusterIP    # Only accessible within the cluster
```

Apply it:

```
kubectl apply -f nginx-service.yaml
kubectl get services
```

Testing within the cluster:

```
# Get the service's ClusterIP
kubectl get service nginx-service

# Create a temporary pod to test from inside the cluster
kubectl run test-pod --rm -it --image=busybox -- sh
# Inside the pod:
```

```
wget -O- http://nginx-service    # Works! DNS resolves 'nginx-service' to the Cluster IP
exit
```

NodePort Service (for local testing)

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080    # Optional: pick a port in the 30000-32767 range
  type: NodePort
```

With kind, you access NodePort services differently. First, port-forward instead:

```
# The easiest way to access a service locally with kind
kubectl port-forward service/nginx-service 8080:80
# Now open http://localhost:8080
```

port-forward creates a temporary tunnel from your local machine to the service. Great for development and debugging.^{[28][29]}

3.6 ConfigMaps and Secrets

The Problem

Hard-coding configuration in your container image is bad. Your image would need to be different for development, staging, and production environments. And you should never bake passwords or API keys into images.^{[30][31][32]}

ConfigMaps — For Non-Sensitive Config

A ConfigMap stores key-value pairs (like environment variables or config file content) that can be injected into Pods.^{[31][32][30]}

Creating a ConfigMap:

```
# Method 1: From literal values
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info \
  --from-literal=MAX_CONNECTIONS=100
```

```
# Method 2: From a file
kubectl create configmap nginx-config --from-file=nginx.conf

# Method 3: From a YAML file (best for git)
```

app-configmap.yaml:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  MAX_CONNECTIONS: "100"
  # You can also store multi-line config files:
  app.properties: |
    server.port=8080
    cache.ttl=3600
    feature.new_ui=true
```

Using ConfigMap as Environment Variables:

```
spec:
  containers:
    - name: my-app
      image: my-app:1.0
      env:
        - name: APP_ENV                # Env var name in container
          valueFrom:
            configMapKeyRef:
              name: app-config          # Name of the ConfigMap
              key: APP_ENV              # Key in the ConfigMap
        # Or inject ALL keys at once:
        - envFrom:
            - configMapRef:
                name: app-config
```

Using ConfigMap as a Mounted File:

```
spec:
  containers:
    - name: my-app
      image: my-app:1.0
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config      # Files appear at this path
  volumes:
    - name: config-volume
      configMap:
        name: app-config
```

Secrets — For Sensitive Data

Secrets are like ConfigMaps but for sensitive values (passwords, API keys, TLS certificates). They're base64-encoded (not encrypted by default, but treated with more care).^[25]^[30]

Creating a Secret:

```
# Method 1: Literal (values are automatically base64-encoded)
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=super_secret_password

# Verify (you'll see base64-encoded values)
kubectl get secret db-credentials -o yaml
```

Secret YAML — note: values must be base64-encoded:

```
echo -n 'super_secret_password' | base64
# Output: c3VwZXJfc2VjcmV0X3Bhc3N3b3Jk
```

db-secret.yaml:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_USER: YWRtaW4= # base64 of "admin"
  DB_PASSWORD: c3VwZXJfc2VjcmV0X3Bhc3N3b3Jk # base64 of "super_secret_password"
```

Using Secrets in a Pod:

```
spec:
  containers:
    - name: my-app
      image: my-app:1.0
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: DB_PASSWORD
```

Security Note: By default, Secrets are only base64-encoded, not encrypted. For production, you should enable etcd encryption or use an external secrets manager like HashiCorp Vault with the External Secrets Operator.^[30]^[25]

3.7 Volumes and Persistent Storage

In Kubernetes, Volumes work similarly to Docker volumes but with more abstraction layers.^[33]
^[34]

The Abstraction Layers

- **PersistentVolume (PV)** — The actual storage resource. Could be a disk on a cloud provider, an NFS share, etc. Created by the cluster admin.
- **PersistentVolumeClaim (PVC)** — A request for storage by a user/application. "I need 10GB of storage." Kubernetes finds a matching PV.
- **StorageClass** — A template that can automatically provision PVs when a PVC is created (dynamic provisioning).

Think of it like booking a hotel:

- StorageClass = Hotel chain with rooms of various sizes
- PVC = Your booking request ("I need a room for 2 nights")
- PV = The actual room assigned to you

Creating a PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce      # RWO: Can be mounted by one node at a time
  resources:
    requests:
      storage: 1Gi        # Request 1 Gigibyte
```

AccessModes:

- **ReadWriteOnce (RWO)** — One node can read and write (typical for databases)
- **ReadOnlyMany (ROX)** — Many nodes can read (config files)
- **ReadWriteMany (RWX)** — Many nodes can read and write (shared filesystems, NFS)

Using the PVC in a Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: postgres-pod
spec:
  containers:
    - name: postgres
      image: postgres:16
```



```

env:
- name: POSTGRES_PASSWORD
  value: "mysecretpassword"
volumeMounts:
- mountPath: /var/lib/postgresql/data
  name: postgres-storage
volumes:
- name: postgres-storage
  persistentVolumeClaim:
    claimName: my-pvc      # Reference the PVC by name

```

3.8 StatefulSets — Databases in Kubernetes

Regular Deployments treat all Pods as interchangeable (cattle). But some apps (databases, message queues) need stable identities: "I am replica #0, my storage is mine, my hostname stays the same."^[34]^[33]

StatefulSet guarantees:

- Stable, unique network identities (pod-0, pod-1, pod-2)
- Stable, persistent storage per pod
- Ordered startup and shutdown^[33]^[34]

StatefulSet Example: PostgreSQL

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: "postgres"    # Headless service name
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:16
        env:
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: DB_PASSWORD
        ports:
        - containerPort: 5432

```

```

    volumeMounts:
      - name: postgres-storage
        mountPath: /var/lib/postgresql/data
    # This creates a PVC for EACH replica automatically!
    volumeClaimTemplates:
      - metadata:
          name: postgres-storage
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 5Gi
    ---
    # Headless Service (required for StatefulSet DNS)
    apiVersion: v1
    kind: Service
    metadata:
      name: postgres
    spec:
      clusterIP: None          # "None" = headless service
      selector:
        app: postgres
      ports:
        - port: 5432

```

With a StatefulSet, the pods get predictable names: postgres-0, postgres-1, etc. Each pod gets its own PVC automatically from volumeClaimTemplates.^{[34][33]}

3.9 Health Checks: Liveness, Readiness, and Startup Probes

Kubernetes needs to know if your application is actually healthy, not just "running." Three types of probes handle this.^{[35][36][37][38]}

Why Probes Matter

Imagine your app starts but then gets stuck in an infinite loop — the process is running but not responding. Or imagine your app takes 30 seconds to start up, and Kubernetes sends traffic before it's ready. Probes solve both problems.^{[38][39][^35]}

The Three Probes

Probe	Question It Answers	Action on Failure
Liveness	Is the app alive and responding?	Restart the container
Readiness	Is the app ready to serve traffic?	Remove from Service load balancer
Startup	Has the app finished starting?	Prevents liveness/readiness from running until startup succeeds

Probe Types

HTTP Probe — Makes an HTTP GET request. Success if response is 200-399.

TCP Probe — Tries to open a TCP connection. Success if it connects.

Command Probe — Runs a command. Success if exit code is 0.

Full Example with All Three Probes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: my-app:1.0
          ports:
            - containerPort: 8080

# Startup probe: Waits up to 5*60=300 seconds for app to start
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 30      # 30 failures allowed
  periodSeconds: 10        # Check every 10 seconds

# Liveness: Restart if app stops responding
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5    # Wait 5s before first check
  periodSeconds: 10        # Check every 10 seconds
  failureThreshold: 3       # Restart after 3 consecutive failures
  timeoutSeconds: 5         # Give the probe 5 seconds to respond

# Readiness: Only send traffic when this passes
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
  failureThreshold: 3
```

Your Flask app should have corresponding endpoints:

```
@app.route('/health')
def health():
    return {'status': 'alive'}, 200

@app.route('/ready')
def ready():
    # Check if database connection works
    try:
        db.execute("SELECT 1")
        return {'status': 'ready'}, 200
    except:
        return {'status': 'not ready'}, 503
```

3.10 Rolling Updates and Rollbacks

One of Kubernetes' most powerful features is the ability to deploy new versions of your app with zero downtime.^{[40][41][42][43]}

How Rolling Updates Work

When you update the image in a Deployment, Kubernetes:

1. Creates a new ReplicaSet with the new version
2. Gradually scales up the new Pods (one or a few at a time)
3. Waits for each new Pod to pass its readiness probe before continuing
4. Scales down the old Pods
5. Removes the old ReplicaSet when empty

No users experience downtime because there are always healthy Pods available.^{[41][42][40]}

Controlling the Update Strategy

```
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # At most 1 extra pod during update (so up to 5 pods total)
      maxUnavailable: 0 # Never go below 4 available pods (zero downtime!)
```

Performing an Update

```
# Method 1: Update the image directly
kubectl set image deployment/nginx-deployment nginx=nginx:1.26

# Method 2: Edit the YAML, change the image, then:
kubectl apply -f nginx-deployment.yaml

# Watch the rollout in real time
kubectl rollout status deployment/nginx-deployment

# Check rollout history
kubectl rollout history deployment/nginx-deployment
```

Rolling Back

If the new version has a bug:

```
# Roll back to the previous version
kubectl rollout undo deployment/nginx-deployment

# Roll back to a specific revision
kubectl rollout undo deployment/nginx-deployment --to-revision=2

# Check history with revision details
kubectl rollout history deployment/nginx-deployment --revision=2
```

Try it yourself:

```
# Update to a non-existent image (will fail)
kubectl set image deployment/nginx-deployment nginx=nginx:999.999.999

# Watch it fail
kubectl rollout status deployment/nginx-deployment
# You'll see the rollout stuck because the image can't be pulled

# Roll back to fix it
kubectl rollout undo deployment/nginx-deployment

# Verify it's healthy again
kubectl get pods
```

3.11 Horizontal Pod Autoscaler (HPA) — Auto-Scaling

The HPA automatically scales the number of Pods in a Deployment based on CPU/memory usage or custom metrics.^{[44][45][46][47]}

How It Works

The HPA monitors the metrics server (a component that collects CPU and memory data from Pods). If average CPU exceeds your target, it adds more Pods. When load drops, it removes Pods, down to a minimum.^{[45][44]}

Step 1 — Install Metrics Server (required for HPA)

kind doesn't include the metrics server by default. Install it:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/metrics-server.yaml
```

For kind, patch it to disable TLS verification (kind uses self-signed certs):

```
kubectl patch deployment metrics-server \
  -n kube-system \
  --type='json' \
  -p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-no-tls-verify"}]'
```

Verify the metrics server is running:

```
kubectl get pods -n kube-system | grep metrics
kubectl top nodes      # Should show CPU/memory usage
kubectl top pods       # Should show pod usage
```

Step 2 — Deploy a Sample App with Resource Requests

HPA requires resource requests to be set:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
spec:
  replicas: 1
  selector:
    matchLabels:
      app: php-apache
  template:
    metadata:
      labels:
        app: php-apache
    spec:
      containers:
        - name: php-apache
          image: registry.k8s.io/hpa-example
          ports:
            - containerPort: 80
          resources:
            requests:
```

```

        cpu: 200m          # Request 0.2 CPU cores
      limits:
        cpu: 500m
    ---
apiVersion: v1
kind: Service
metadata:
  name: php-apache
spec:
  selector:
    app: php-apache
  ports:
    - port: 80

```

Step 3 — Create the HPA

```

kubectl autoscale deployment php-apache \
  --cpu-percent=50 \
  --min=1 \
  --max=10

```

This says: "Keep CPU usage around 50%. Minimum 1 pod, maximum 10 pods."

Or in YAML:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50      # Target: 50% CPU utilization

```

Step 4 — Generate Load and Watch It Scale

```

# Terminal 1: Watch HPA
kubectl get hpa -w

# Terminal 2: Generate load
kubectl run load-generator \

```

```
--rm -i \  
--image=busybox \  
--restart=Never \  
-- /bin/sh -c "while sleep 0.01; do wget -q -O- http://php-apache; done"
```

Watch Terminal 1 — you'll see the CPU go up and new Pods get created. Stop the load generator (Ctrl+C) and watch the Pods scale back down.^[46]^[44]^[45]

3.12 Networking Deep Dive — Ingress and Network Policies

Ingress — The Front Door

A Service of type LoadBalancer or NodePort exposes one service to the outside. Ingress is smarter — it's a single entry point that routes traffic to multiple services based on the URL path or hostname.^[48]

Think of Ingress like a receptionist in a large office building. One front door (one IP address), but the receptionist routes "If you're here for finance, go to floor 3. For engineering, go to floor 5."^[48]

Step 1 — Install an Ingress Controller (nginx ingress).

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy
```

Wait for it to be ready:

```
kubectl wait --namespace ingress-nginx \  
--for=condition=ready pod \  
--selector=app.kubernetes.io/component=controller \  
--timeout=90s
```

Step 2 — Create the Ingress resource.

```
apiVersion: networking.k8s.io/v1  
kind: Ingress  
metadata:  
  name: my-app-ingress  
  annotations:  
    nginx.ingress.kubernetes.io/rewrite-target: /  
spec:  
  rules:  
  - host: myapp.local          # Route based on hostname  
    http:  
      paths:  
      - path: /frontend  
        pathType: Prefix  
        backend:  
          service:  
            name: frontend-service
```



```

      port:
        number: 80
- path: /api
  pathType: Prefix
  backend:
    service:
      name: backend-service
      port:
        number: 8080

```

Requests to `myapp.local/frontend` → `frontend-service`. Requests to `myapp.local/api` → `backend-service`.^[48]

Network Policies — The Firewall Inside Your Cluster

By default, all Pods in a Kubernetes cluster can communicate with all other Pods freely. This is a security risk: if an attacker compromises one Pod, they can reach every other Pod.^{[49][50][51]}

Network Policies are your internal firewall. They let you say: "Only the frontend can talk to the backend. Only the backend can talk to the database. The database cannot initiate connections to anything."^{[50][49]}

Step 1 — Understand the default.

Without any Network Policies: all traffic allowed in all directions.

When you apply a Network Policy to a Pod: only traffic explicitly allowed in the policy is permitted. All other traffic is denied.^{[51][49]}

Step 2 — Create a default deny-all policy.

This is the recommended starting point for any production cluster:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: default
spec:
  podSelector: {}          # Empty selector = applies to ALL pods in namespace
  policyTypes:
    - Ingress
    - Egress

```

After applying this, every Pod in the `default` namespace is isolated. Now you add specific allow rules.^{[49][50][51]}

Step 3 — Allow frontend → backend traffic.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy

```

```

metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend          # This policy applies to backend pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend      # Only allow traffic FROM frontend pods
      ports:
      - protocol: TCP
        port: 8080

```

Step 4 — Allow backend → database traffic.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-to-database
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: backend
      ports:
      - protocol: TCP
        port: 5432

```

Step 5 — Test your policies.

```

# Create test pods
kubectl run frontend --image=busybox -l app=frontend -- sleep 3600
kubectl run backend  --image=busybox -l app=backend  -- sleep 3600
kubectl run database --image=busybox -l app=database -- sleep 3600

# Frontend should be able to reach backend
kubectl exec frontend -- wget -qO- http://backend:8080 # Should work

# Frontend should NOT be able to reach database
kubectl exec frontend -- wget -qO- http://database:5432 # Should fail/timeout

```

3.13 RBAC — Security and Access Control

RBAC (Role-Based Access Control) controls who can do what in your Kubernetes cluster. It answers: "Can user X perform action Y on resource Z?"^{[52][53][54][55][56]}

Key Concepts

- **Role** — Defines a set of permissions within a namespace
- **ClusterRole** — Like a Role, but applies across all namespaces (or for cluster-wide resources)
- **RoleBinding** — Binds a Role to a user, group, or ServiceAccount within a namespace
- **ClusterRoleBinding** — Binds a ClusterRole cluster-wide
- **ServiceAccount** — An identity for Pods (like a user account, but for applications)

The Principle of Least Privilege

Every user and every Pod should have only the minimum permissions needed. Nothing more.^{[53][54]}

Step 1 — Create a ServiceAccount for Your App

When your app needs to call the Kubernetes API (e.g., for service discovery), it should use a ServiceAccount with minimal permissions:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: default
```

```
kubectl apply -f serviceaccount.yaml
```

Step 2 — Create a Role

This Role allows reading Pods and ConfigMaps in the default namespace:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: ["" ]          # "" means core API group (pods, services, etc.)
  resources: ["pods", "configmaps"]
  verbs: ["get", "list", "watch"] # Only read operations
# This role CANNOT create, update, or delete pods
```

Step 3 — Bind the Role to the ServiceAccount

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-app-sa           # Bind to our service account
  namespace: default
roleRef:
  kind: Role
  name: pod-reader          # The role defined above
  apiGroup: rbac.authorization.k8s.io
```

Step 4 — Use the ServiceAccount in a Deployment

```
spec:
  template:
    spec:
      serviceAccountName: my-app-sa  # Add this line
      containers:
      - name: my-app
        image: my-app:1.0
```

Step 5 — Check Permissions

```
# Check if my-app-sa can list pods
kubectl auth can-i list pods \
  --as=system:serviceaccount:default:my-app-sa

# Check if it can delete pods (should be no)
kubectl auth can-i delete pods \
  --as=system:serviceaccount:default:my-app-sa
```

Creating a Developer User

For a developer named "alice" who should only read pods in the dev namespace:

```
# Step 1: Generate a private key
openssl genrsa -out alice.key 2048

# Step 2: Create a certificate signing request
openssl req -new -key alice.key -out alice.csr -subj "/CN=alice/O=dev-team"

# Step 3: Create a Kubernetes CertificateSigningRequest
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
```

```

metadata:
  name: alice
spec:
  request: $(cat alice.csr | base64 | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages:
    - client auth
EOF

# Step 4: Approve the CSR
kubectl certificate approve alice

# Step 5: Get the signed certificate
kubectl get csr alice -o jsonpath='{.status.certificate}' | base64 -d > alice.crt

# Step 6: Add to kubeconfig
kubectl config set-credentials alice \
  --client-certificate=alice.crt \
  --client-key=alice.key

kubectl config set-context alice-context \
  --cluster=kind-dev-cluster \
  --user=alice

```

3.14 Helm — The Kubernetes Package Manager

Deploying a complex application on Kubernetes might involve 10 or more YAML files (Deployments, Services, ConfigMaps, Secrets, Ingress, etc.). Helm manages all of these as a single package called a **chart**.^{[57][58]}

Think of Helm like `apt` or `brew` — but for Kubernetes applications. One command to install everything, one command to upgrade, one command to delete.

Installing Helm

macOS:

```

brew install helm
helm version

```

Linux:

```

curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version

```

Using Existing Helm Charts

```
# Add a chart repository (like adding a software source)
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search available charts
helm search repo nginx
helm search repo postgres

# Install a chart
helm install my-postgres bitnami/postgresql \
  --set auth.postgresPassword=mysecretpassword \
  --set primary.persistence.size=5Gi

# List installed releases
helm list

# Upgrade a release
helm upgrade my-postgres bitnami/postgresql \
  --set auth.postgresPassword=newpassword

# Rollback to previous version
helm rollback my-postgres 1

# Uninstall (removes all Kubernetes resources)
helm uninstall my-postgres
```

Creating Your Own Helm Chart

```
# Create a chart skeleton
helm create my-app
```

This creates:

```
my-app/
├── Chart.yaml           # Chart metadata
├── values.yaml          # Default configuration values
└── templates/          # Kubernetes YAML templates
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    └── ...
```

values.yaml (your defaults):

```
image:
  repository: my-flask-app
  tag: "1.0"
  pullPolicy: IfNotPresent
```

```
replicaCount: 2

service:
  type: ClusterIP
  port: 5000

resources:
  limits:
    cpu: 200m
    memory: 256Mi
  requests:
    cpu: 100m
    memory: 128Mi
```

templates/deployment.yaml uses Go template syntax:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-deployment
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Installing your chart:

```
# Install with default values
helm install my-release ./my-app

# Install with overridden values
helm install my-release ./my-app --set replicaCount=5

# Install with a custom values file
helm install my-release ./my-app -f production-values.yaml

# Preview the generated YAML (dry run)
helm template my-release ./my-app

# Validate the chart
helm lint ./my-app
```

Part 3 — Real-World Project

Deploying a Full Three-Tier Application

Let's tie everything together by deploying a real application: a React frontend, a Node.js API backend, and a PostgreSQL database — all on Kubernetes, with proper configuration, secrets, and networking.

Project Structure

```
k8s-fullstack/
├── frontend/
│   ├── Dockerfile
│   └── (React app files)
├── backend/
│   ├── Dockerfile
│   └── (Node.js API files)
└── k8s/
    ├── namespace.yaml
    ├── secrets.yaml
    ├── configmap.yaml
    ├── postgres-statefulset.yaml
    ├── postgres-service.yaml
    ├── backend-deployment.yaml
    ├── backend-service.yaml
    ├── frontend-deployment.yaml
    ├── frontend-service.yaml
    ├── ingress.yaml
    └── network-policy.yaml
```

Step 1 — Create the Namespace

namespace.yaml:

```
apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:
    name: production
```

```
kubectl apply -f k8s/namespace.yaml
kubectl config set-context --current --namespace=production
```

Step 2 — Create Secrets and ConfigMaps

secrets.yaml:

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
```



```
namespace: production
type: Opaque
stringData:          # stringData auto-encodes to base64
  DB_PASSWORD: "SuperSecretPassword123"
  JWT_SECRET: "your-jwt-secret-key-here"
```

configmap.yaml:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: production
data:
  DB_HOST: "postgres-service"
  DB_PORT: "5432"
  DB_NAME: "myapp"
  DB_USER: "appuser"
  NODE_ENV: "production"
  API_URL: "http://backend-service:3000"
```

```
kubectl apply -f k8s/secrets.yaml
kubectl apply -f k8s/configmap.yaml
```

Step 3 — Deploy PostgreSQL

postgres-statefulset.yaml:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: production
spec:
  serviceName: postgres-service
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          env:
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: app-config
```

```

        key: DB_NAME
      - name: POSTGRES_USER
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: DB_USER
      - name: POSTGRES_PASSWORD
        valueFrom:
          secretKeyRef:
            name: app-secrets
            key: DB_PASSWORD
    ports:
      - containerPort: 5432
    livenessProbe:
      exec:
        command:
          - pg_isready
          - -U
          - appuser
        initialDelaySeconds: 30
        periodSeconds: 10
    readinessProbe:
      exec:
        command:
          - pg_isready
          - -U
          - appuser
        initialDelaySeconds: 5
        periodSeconds: 5
    volumeMounts:
      - name: postgres-storage
        mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: postgres-storage
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
  namespace: production
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
  clusterIP: None      # Headless service for StatefulSet

```

Step 4 — Deploy the Backend

backend-deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: production
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: my-backend:1.0
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: app-config
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secrets
                  key: DB_PASSWORD
      livenessProbe:
        httpGet:
          path: /health
          port: 3000
        initialDelaySeconds: 30
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /ready
          port: 3000
        initialDelaySeconds: 10
        periodSeconds: 5
      resources:
        requests:
          cpu: 100m
          memory: 128Mi
        limits:
          cpu: 500m
          memory: 512Mi
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: backend-service
  namespace: production
spec:
  selector:
    app: backend
  ports:
    - port: 3000
```

Step 5 — Apply Network Policies

network-policy.yaml:

```
# Deny all traffic by default
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
---
# Frontend can receive external traffic and call backend
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - {}          # Allow from anywhere (ingress controller)
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: backend
        ports:
          - port: 3000
---
# Backend can receive from frontend, call postgres, and DNS
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: production
spec:
  podSelector:
```

```

    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: postgres
  ports:
  - port: 5432
  - to: # Allow DNS resolution
    - namespaceSelector: {}
  ports:
  - port: 53
    protocol: UDP

```

Step 6 — Deploy Everything

```

kubectl apply -f k8s/
kubectl get all -n production

# Watch everything come up
kubectl get pods -n production -w

```

Step 7 — Verify

```

# Check all pods are running
kubectl get pods -n production

# Check services
kubectl get services -n production

# Check logs
kubectl logs deployment/backend -n production

# Port-forward to test locally
kubectl port-forward service/backend-service 3000:3000 -n production
# Test: curl http://localhost:3000/health

```

Part 4 — DevSecOps Context

Where Security Fits In

DevSecOps means integrating security at every stage, not just at the end.^{[4][8][^3]}

Security at the Docker Layer

1. **Scan images in your build pipeline:** Every time you build an image, scan it with Trivy or Docker Scout before pushing it to the registry.^{[3][21]}
2. **Use minimal base images:** alpine and distroless images have fewer packages = fewer vulnerabilities.^{[21][13]}
3. **Never run as root:** Always set a USER directive.^{[14][21][^13]}
4. **Sign your images:** Docker Content Trust ensures images haven't been tampered with.
5. **Use .dockerignore:** Prevent credentials and secrets from entering images.^[^13]

Security at the Kubernetes Layer

1. **RBAC everywhere:** Apply least privilege to all ServiceAccounts and users.^{[52][53][^54]}
2. **Network Policies:** Default deny, then allow only what's needed.^{[49][50]}
3. **Pod Security Standards:** Kubernetes has built-in policies (Privileged/Baseline/Restricted) to prevent dangerous Pod configurations.
4. **Secrets management:** For production, use external solutions (HashiCorp Vault + External Secrets Operator) instead of Kubernetes Secrets stored in etcd.^[^25]
5. **Audit logging:** Enable audit logs on the API server to track who did what.

The CI/CD Security Pipeline

A mature DevSecOps pipeline looks like:

```
Code Push
  ↓
Static Analysis (SAST) – Find code bugs
  ↓
Dependency Scanning – Check for vulnerable libraries
  ↓
Docker Build
  ↓
Image Scanning (Trivy/Snyk) – Find OS/library CVEs
  ↓
Push to Registry (only if clean)
  ↓
Deploy to Kubernetes
  ↓
Runtime Security (Falco) – Detect unusual behavior
```



Monitoring & Alerting (Prometheus/Grafana)

Tools to Learn After This Guide

Category	Tool	Purpose
CI/CD	GitHub Actions / Jenkins	Automate build and deploy pipeline
Image Scanning	Trivy, Snyk, Docker Scout	Find vulnerabilities in images
Secrets Management	HashiCorp Vault	Centralized secrets management
Policy Enforcement	Kyverno, OPA/Gatekeeper	Enforce security policies on K8s resources
Runtime Security	Falco	Detect threats at runtime
Monitoring	Prometheus + Grafana	Metrics and alerting
GitOps	ArgoCD	Declarative, Git-driven deployments
Service Mesh	Istio / Linkerd	mTLS, advanced traffic control

Quick Reference Cheat Sheet

Docker Commands

```
# Lifecycle
docker run -d -p 8080:80 --name myapp nginx:1.25
docker ps -a
docker stop myapp
docker start myapp
docker rm myapp
docker rm -f myapp          # Force remove running container

# Images
docker build -t myapp:1.0 .
docker images
docker pull nginx:1.25
docker rmi nginx:1.25
docker image prune -a

# Logs & Debugging
docker logs myapp
docker logs -f myapp
docker exec -it myapp bash
docker inspect myapp
docker stats

# Volumes
docker volume create mydata
docker run -v mydata:/data nginx
docker volume ls
docker volume rm mydata
```

```
# Networks
docker network create mynet
docker run --network mynet nginx
docker network ls

# Compose
docker compose up -d
docker compose down
docker compose down -v      # Also remove volumes
docker compose logs -f
docker compose exec web bash
docker compose ps
docker compose restart web
```

kubectl Commands

```
# Cluster Info
kubectl cluster-info
kubectl get nodes
kubectl top nodes

# Working with Resources
kubectl apply -f file.yaml
kubectl delete -f file.yaml
kubectl get pods
kubectl get pods -n kube-system
kubectl get all
kubectl get all -A          # All namespaces

# Debugging
kubectl describe pod mypod
kubectl logs mypod
kubectl logs -f mypod
kubectl exec -it mypod -- bash
kubectl port-forward pod/mypod 8080:80
kubectl port-forward service/myservice 8080:80

# Deployments
kubectl get deployments
kubectl scale deployment myapp --replicas=5
kubectl set image deployment/myapp container=image:v2
kubectl rollout status deployment/myapp
kubectl rollout undo deployment/myapp
kubectl rollout history deployment/myapp

# Context & Namespace
kubectl config get-contexts
kubectl config use-context kind-dev-cluster
kubectl config set-context --current --namespace=mynamespace

# Secrets & Config
kubectl get configmaps
kubectl get secrets
kubectl describe secret mysecret
```



```
kubectl get secret mysecret -o jsonpath='{.data.password}' | base64 -d
```

```
# RBAC
```

```
kubectl get roles
```

```
kubectl get rolebindings
```

```
kubectl auth can-i list pods --as=system:serviceaccount:default:mysa
```

YAML Quick Templates

Namespace:

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
```

Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: my-container
      image: nginx:1.25
```

Deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: my-image:1.0
          ports:
            - containerPort: 8080
```

Service:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  KEY: "value"
  ANOTHER_KEY: "another_value"
```

Secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
stringData:
  password: "mysecretpassword"
```

References

1. [Docker Beginner to Advanced 2025: The Complete Guide for ...](#) - Install and configure Docker on Windows, macOS, and Linux. Understand core Docker concepts like cont...
2. [Docker for Beginners: A Practical Guide to Containers](#) - This beginner-friendly tutorial covers the essentials of containerization, helping you build, run, a...
3. [Complete DevSecOps Roadmap for Beginners](#) - DevSecOps Learning Roadmap Overview · Fundamentals · Development Basics · DevOps Core Skills · Cloud...
4. [DevOps & DecSecOps Roadmap \[From beginner to an expert\]](#) - Docker and Kubernetes are two important containerization tools. Kubernetes offers container orchestr...
5. [Learn Kubernetes Basics](#) - 14 Jan 2026 — ObjectivesThis tutorial provides a walkthrough of the basics of the Kubernetes c...
6. [Kubernetes Documentation](#) - Learn how to use Kubernetes. Look up common tasks and how to perform them using a short sequence of ...

7. [A Beginner's Guide to Kubernetes](#) - In this article, we will learn how Kubernetes is a system of promises, and that every piece of it is...
8. [The DevSecOps roadmap that takes you from Linux basics ...](#) - The DevSecOps roadmap that takes you from Linux basics to Kubernetes in production, with real comman...
9. [Writing a Dockerfile](#) - Try it out. Just as you saw with the previous example, a Dockerfile typically follows these steps: D...
10. [Docker Desktop for macOS Setup and Tips](#) - When you're getting started with Docker and Kubernetes, you'll likely want a local setup. This video...
11. [Install Docker Engine](#) - This section describes how to install Docker Engine on Linux, also known as Docker CE. Docker Engine...
12. [Building best practices](#) - Building best practices · Use multi-stage builds · Choose the right base image · Rebuild your images...
13. [Top 8 Docker Best Practices for using Docker in Production](#) ✓ - Use an official and verified Docker image as a base image, whenever available. · Use specific Docker...
14. [dnaprawa/dockerfile-best-practices](#) - This repository contains some best-practices for writing Dockerfiles. Most of them are coming from m...
15. [Best Practices and Tips for Writing a Dockerfile](#) - You should utilize a container scanning engine as a best-practice step to avoid issues slipping into...
16. [Volumes](#) - Learn how to create, manage, and use volumes instead of bind mounts for persisting data generated an...
17. [Docker Volumes explained in 6 minutes](#) - In this video we're gonna learn about docker volumes in a natural docker volumes are used for data p...
18. [What Is Docker Volume?](#) - Docker Volumes are persistent storage mechanisms managed by Docker that allow containers to retain d...
19. [Docker Compose](#) - How to Use Docker Compose? · Step 1: Create Project Directory · Step 2: Create API · Step 3: Build P...
20. [Docker Compose - What is It, Example & Tutorial](#) - What is Docker Compose? See a getting started tutorial for beginners with file structure, commands, ...
21. [Top 21 Dockerfile best practices for container security](#) - Since RUN, COPY, ADD, and other instructions create a new container layer, grouping multiple command...
22. [Quick Start - kind - Kubernetes](#) - This guide covers getting started with the kind command. If you are having problems please see the k...
23. [kind - Kubernetes](#) - kind is a tool for running local Kubernetes clusters using Docker container “nodes”. kind was primar...
24. [Setting Up Local Kubernetes Cluster with Kind](#) - Setting up Kubernetes locally using Kind is a crucial step in understanding the core components and ...
25. [Day-5 | Kubernetes Security Tutorial - RBAC, Network Policies ...](#) - In today's video we will cover six different topics in the video Starting with what are name spaces ...
26. [Demystifying Kubernetes: Pods, Deployments, and Services](#) - Here's an example of a Deployment definition in YAML format ... Here's an example of a Service defin...

27. [Kubernetes Deployment YAML Explained with Examples](#) - In this blog we'll cover what a Kubernetes Deployment YAML file is, how it works & how to write a YA...
28. [How to Create Deployments and Services in Kubernetes?](#) - Kubernetes is a container orchestration tool that helps with the deployment and management of contain...
29. [Connecting Applications with Services](#) - A Kubernetes Service is an abstraction which defines a logical set of Pods running somewhere in your...
30. [Kubernetes ConfigMaps & Secrets Explained Clearly](#) - Learn how Kubernetes manages app settings and sensitive data securely using ConfigMaps and Secrets w...
31. [Kubernetes ConfigMap & Secrets](#) - Master Kubernetes ConfigMap usage with real examples. Learn config separation, mounting and best pra...
32. [ConfigMaps](#) - A ConfigMap allows you to decouple environment-specific configuration from your container images, so...
33. [StatefulSets](#) - If you want to use storage volumes to provide persistence for your workload, you can use a StatefulS...
34. [Kubernetes StatefulSets Overview](#) - When you deploy a StatefulSet, K8s will assign each replica its own state (volumes) and guarantee th...
35. [Kubernetes Readiness Probe: Tutorial & Examples](#) - A liveness probe is used to determine if a container is still running. If the probe fails, the conta...
36. [Configure Liveness, Readiness and Startup Probes](#) - This page shows how to configure liveness, readiness and startup probes for containers. For more inf...
37. [Kubernetes Readiness Probes - Examples & Common ...](#) - This post discusses how readiness probes can be configured and how to prevent common pitfalls.
38. [Guide to Kubernetes Liveness Probes with Examples](#) - Learn how Kubernetes liveness probes work, different types, configurations, and best practices to ke...
39. [Readiness vs liveness probes: How to set them up and ...](#) - Readiness probes are designed to let Kubernetes know when your app is ready to serve traffic. Kubern...
40. [Rolling Updates and Rollbacks in Kubernetes: Managing ...](#) - Rollbacks are like an "undo" button - they revert application back to the safe, stable previous vers...
41. [Kubernetes Rolling Update: Deploy Without Service ...](#) - Step 1: Understand the Rolling Update Mechanism. A rolling update is a Kubernetes progressive update...
42. [Performing a Rolling Update](#) - A rolling update allows a Deployment update to take place with zero downtime. It does this by increm...
43. [Kubernetes Rolling Deployment: A Practical Guide](#) - To perform a rolling update, you first need to define a Deployment object in your Kubernetes cluster...
44. [Scale pod deployments with Horizontal Pod Autoscaler](#) - This example is based on the Horizontal Pod autoscaler walkthrough in the Kubernetes documentation. ...
45. [Autoscaling Workloads](#) - There is a walkthrough tutorial of configuring a HorizontalPodAutoscaler for a Deployment. Scaling w...
46. [HorizontalPodAutoscaler Walkthrough](#) - This document walks you through an example of enabling HorizontalPodAutoscaler to automatically mana...

47. [Kubernetes HPA \[Horizontal Pod Autoscaler\] Guide](#) - Example: How to set up Kubernetes HPA · HPA limitations and gotchas · Kubernetes HPA best practices....
48. [Ingress](#) - The Ingress concept lets you map traffic to different backends based on rules you define via the Kub...
49. [Kubernetes Network Policy - Guide with Examples](#) - Learn what are Network Policies in Kubernetes, how to implement them and best practices. See example...
50. [What is Kubernetes Network Policy?](#) - ARMO - Kubernetes Network Policy determines how cloud cluster resources such as pods are segmented. The pol...
51. [networkpolicy/tutorial: Kubernetes Network Policy Tutorial](#) - Ingress and egress are terms used in networking to describe traffic direction. Ingress refers to net...
52. [Using RBAC Authorization](#) - To enable RBAC, start the API server with the --authorization-config flag set to a file that include...
53. [Guide to Kubernetes RBAC - Example & Best Practices](#) - How to set up RBAC in Kubernetes - example · 1. Check whether RBAC is enabled · 2. Create a Service ...
54. [Kubernetes RBAC Explained: Key Concepts and Best ...](#) - What is Kubernetes role-based access control (RBAC)? Learn the basics of RBAC in Kubernetes, and vie...
55. [Kubernetes RBAC Authorization: The Ultimate Guide](#) - Kubernetes RBAC Configuration Examples. Here's a concrete example. To grant a user named "bob" read ...
56. [Kubernetes RBAC: the Complete Best Practices Guide](#) - ARMO - Master RBAC Kubernetes: Manage roles, permissions, and service accounts to secure cluster access. Ex...
57. [Helm Chart Tutorial: A Simple Guide for Beginners](#) - Learn how to create a Helm chart with our easy-to-follow Helm Charts Tutorial. This guide covers str...
58. [Kubernetes Helm charts: The basics and a quick tutorial](#) - These charts act as templates, which help in defining, ...